EC527 Final Project Report: GMM Optimization

Kohki Hatori and Mimi Paule

April 25, 2026

Contents

1 Description of the Problem **2**

2 Serial Code and Algorithm **2**

3 Performance Profiling & Bottlenecks **2**

4 Data Structures & Memory **3**

5 Parallel Modifications **3**

6 Partitioning Strategy **4**

7 Optimizations Overview **4**

8 Experiments and Results **4**

8.1 OpenMP Scaling Analysis ($D = 32, K = 8$) 5

8.2 Comparison of Serial, OpenMP, and CUDA ($D = 32, K = 8$) 5

8.3 CPU vs GPU Scaling Analysis (Broad Grid) 5

8.4 Architectural Performance Factors 7

8.5 3D Surface Analysis 7

9 Future Work for GPU Acceleration **7**

10 Conclusion **8**

1 Description of the Problem

This project focuses on the performance optimization of the Gaussian Mixture Model (GMM) training process. GMM is a robust probabilistic model used extensively in machine learning, signal processing, and data analysis for tasks such as clustering, anomaly detection, and density estimation. Training a GMM entails fitting a dataset of N points in a D-dimensional space to K distinct Gaussian distributions by estimating their corresponding weights, means, and covariance matrices.

The parameter estimation is heavily reliant on the iterative Expectation-Maximization (EM) algorithm, which alternates between two computationally rigorous phases:

- **E-Step (Expectation):** Evaluates the multivariate Gaussian probability density function for every data point across all K clusters to compute posterior probabilities (responsibilities).
- **M-Step (Maximization):** Aggregates the responsibilities across the dataset to update the model parameters (weights, means, and covariances).

As dataset sizes (N) and dimensionalities (D) increase, the EM algorithm becomes highly computationally and memory intensive due to continuous data passes and complex linear algebra operations (such as Cholesky decompositions for covariance matrix inversion). The primary objective of this project is to analyze the performance bottlenecks of a serial GMM implementation and accelerate the algorithm by utilizing parallel computing paradigms, specifically OpenMP for multi-core CPUs and CUDA for GPU architectures.

2 Serial Code and Algorithm

The base serial implementation fundamentally loops through a predefined number of iterations or until convergence, executing the Expectation (E) and Maximization (M) steps sequentially.

- **Expectation Step:** For every data point x_i ($1 \leq i \leq N$) and every mixture component k ($1 \leq k \leq K$), the algorithm computes the continuous multidimensional Gaussian probability. This involves first performing a Cholesky decomposition to invert the covariance matrix, then computing the Mahalanobis distance by subtracting the mean vector, multiplying by the inverse covariance matrix, and calculating the exponential. The results are normalized to determine the responsibility $\gamma_{i,k}$, representing the probabilistic assignment of point x_i to cluster k.
- **Maximization Step:** The responsibilities are used to recalculate the parameters of the distributions. First, the algorithm computes the total effective weight for each cluster. Next, it performs a full sweep through the dataset to compute the new mean vectors. Finally, it executes another sweep to update the covariance matrices using the newly found means and the responsibility weights.

Algorithmic Complexity: The computational complexity per EM iteration is driven by varying operations depending on the dataset grid. During the E-step, computing the multivariate Gaussian PDF requires a Cholesky decomposition and subsequent matrix inversion of the covariance matrix for each of the K clusters, an operation that scales dramatically as $\mathcal{O}(K \cdot D^3)$. Furthermore, computing the Mahalanobis distance for all N points and updating the covariances in the M-step dictates vector-matrix multiplications scaling as $\mathcal{O}(N \cdot K \cdot D^2)$.

Therefore, for small dimensionality ($D \leq 16$), the $\mathcal{O}(N \cdot K \cdot D^2)$ memory-bound operations dominate the execution. As D increases, the $\mathcal{O}(K \cdot D^3)$ Cholesky matrix inversions cause the workload to become immensely compute-bound, representing the hardware inflection point between CPU caching/vectorization advantages and raw GPU FLOP throughput. Over I iterations, the total computational complexity is bounded by $\mathcal{O}(I \cdot (K \cdot D^3 + N \cdot K \cdot D^2))$.

3 Performance Profiling & Bottlenecks

Performance profiling on the serial execution (using tools like `gprof` or Xcode Instruments) reveals that the overwhelming majority of the runtime is concentrated within two heavily nested loops: the Gaussian probability calculation in the E-step, and the covariance matrix calculation in the M-step.

When evaluating the arithmetic intensity (the ratio of floating-point operations to bytes of memory accessed), GMM training is notoriously **memory-bandwidth bound** rather than strictly compute-bound.

For every iteration:

1. The E-step reads the entire $N \times D$ dataset matrix.
2. The M-step (means calculation) reads the $N \times D$ dataset matrix again along with the $N \times K$ responsibilities.
3. The M-step (covariance calculation) reads the $N \times D$ dataset matrix a third time to calculate the outer products.

This data-access paradigm forces the processor to stream the massive $N \times D$ memory footprint from main DRAM three separate times per iteration. Consequently, parallel optimization strategies must account for memory movement, as compute pipelines often stall waiting for variable fetches rather than arithmetic exhaustion.

4 Data Structures & Memory

To support the EM algorithm, the implementation relies on several primary data structures. To ensure cache locality and avoid pointer chasing, these are allocated as contiguous 1D memory blocks representing 2D or 3D matrices in row-major order:

- **Dataset (`data`):** An $N \times D$ matrix containing the input features.
- **Means (`means`):** A $K \times D$ matrix tracking the centroid of each Gaussian component.
- **Covariances (`covariances`):** A $K \times D \times D$ tensor representing the covariance matrix for each component.
- **Responsibilities (`responsibilities`):** An $N \times K$ matrix storing the probability of each point belonging to each class.
- **Inverse Covariances:** A pre-calculated $K \times D \times D$ tensor to avoid repeated matrix inversions inside the inner loop.

Memory Reference Pattern: The memory access pattern across the dataset is largely sequential. During the E and M-steps, the outermost loop iterates over the dataset N, and the inner loops iterate over K and D. Because the dataset is laid out such that the D dimensions for a single point are perfectly contiguous, the CPU accesses the memory sequentially, step-by-step.

This sequential read pattern is highly cache-friendly. Modern CPU hardware prefetchers can easily detect the linear access stride and proactively load the memory lines into L1 and L2 caches ahead of the execution core, efficiently masking latency. However, for extremely large values of N, the full dataset capacity easily exceeds the L3 cache, causing unavoidable capacities misses and cyclic cache flushing over the iterations.

5 Parallel Modifications

To transition the serial algorithm into parallel applications capable of producing mathematically identical results asynchronously, several macro structural modifications were enacted to resolve deep-seated data dependencies and prevent race conditions.

- **OpenMP (CPU):** In the baseline serial code, updating the global distribution parameters (means and covariances) during the M-step causes massive data contention. To safely parallelize this, thread-local storage was introduced. Every active thread allocates a private computational space (e.g., `local_weight`, `local_mean`, and `local_covariance`). Threads autonomously accumulate partial sums from their designated data chunks into these private structures lock-free. After the intensive parallel loops conclude, a singular `#pragma omp critical` syncing step securely aggregates the scattered local results into the unified global distribution parameters.
- **CUDA (GPU):** The GPU adaptation requires translating deeply nested iteration loops into flattened kernel launches. Instead of looping through instances sequentially, the outer dataset loop iterating over N is mapped to the GPU grid layout. Each CUDA thread calculates the probabilities and continuous responsibilities exclusively for a single assigned data point. To update the global parameters without memory corruption, the kernels rely on `atomicAdd()` operations to serialize concurrent floating-point additions effectively.

6 Partitioning Strategy

Determining how the immense data workload is distributed directly influences the efficiency of the implementation.

- **OpenMP Partitioning:** Since the algorithmic iterations over the N points in the dataset are operationally independent during the E-step and early M-step, a 1D flat partitioning approach is utilized. By defining `#pragma omp parallel for schedule(static)`, the $N \times D$ data matrix is evenly sectioned out directly to the active hardware threads. For an execution configuration of $N = 500,000$ points running across 24 threads, each underlying core is assigned to process precisely $\approx 20,833$ contiguous points, ensuring absolute L1 cache lineage continuity.

- **CUDA Partitioning:** The CUDA grid is similarly modeled leveraging 1D mapping logic. The absolute thread index (`tid = blockIdx.x * blockDim.x + threadIdx.x`) acts directly as the designated pointer to an individual point x_i. Grid dimensions are dynamically evaluated using ceiling integer division (e.g., $\lceil N/256 \rceil$), sorting the workload into parallel thread-blocks of size 256. This layout forces Streaming Multiprocessors to access the contiguous feature arrays in memory-coalesced 32-thread warp executions.

7 Optimizations Overview

Once parallel correctness was achieved, several underlying hardware optimizations were layered onto the structure to enhance memory bandwidth utilization and floating-point throughput.

OpenMP CPU Optimizations:

1. **SIMD Vectorization Alignments:** The defined mathematical baseline ($D = 32$) acts as a highly optimized parameter metric for contemporary CPUs. Since 32 floating-point values resolve to exactly 128 bytes, they efficiently saturate AVX2 (256-bit) and AVX-512 vector lengths across execution cycles. Utilizing the ‘gcc‘ compiler flags (‘-O3 -ffast-math‘), innermost D dimension loops natively compile down to singular SIMD vectorized assembly instructions without requiring manual intrinsic injections.

2. **Lock-Free Execution Cycles:** Replacing persistent reduction locks or atomic operations inside the loop body with the isolated ‘local_covariance‘ arrays bypasses immense OS thread-contention and lock latency across the 24 cores.

CUDA GPU Optimizations & Challenges:

1. **Shared Memory Tiling Mechanisms:** To deflect the severe latency involved in repeatedly accessing configuration tensors (like the `means` array) directly from standard HBM2 global memory, evaluating blocks proactively load subsets of cluster constraints directly into local SM `__shared__` memory during initial execution stages, streamlining execution loops directly from SRAM.

2. **Challenge: Heavy Atomic Contention:** During the CUDA M-step reduction phases, compiling the global matrices requires significant executions of `atomicAdd` in global or shared memory. In parameter ranges operating high cluster counts (K), increasing volumes of threads attempt to write over the exact identical memory destinations simultaneously for probability bins. This hardware bottleneck mandates immense thread serialization queuing logic, occasionally forcing the GPU implementation to underperform its highly vectored CPU equivalent.

8 Experiments and Results

In this section, we evaluate the performance of our serial, OpenMP, and CUDA implementations. We first examine the scaling behavior of OpenMP threads for a fixed model complexity, followed by a broad parameter grid search across varying D and K.

8.1 OpenMP Scaling Analysis ($D = 32, K = 8$)

We performed a scaling analysis of the OpenMP implementation by varying the number of threads from 2 to 32. For this experiment, the dimensionality was fixed at $D = 32$ and the number of clusters at $K = 8$, while the dataset size N was scaled from 10,000 to 3,000,000 points.

As illustrated in Figure 1, the execution time scales effectively with the number of threads. The 32-thread configuration consistently provides the lowest wall time, completing the 3M point workload in 3.88 seconds, compared to 25.73 seconds for the 2-thread configuration. This demonstrates high parallel efficiency and effective utilization of the multi-core architecture.

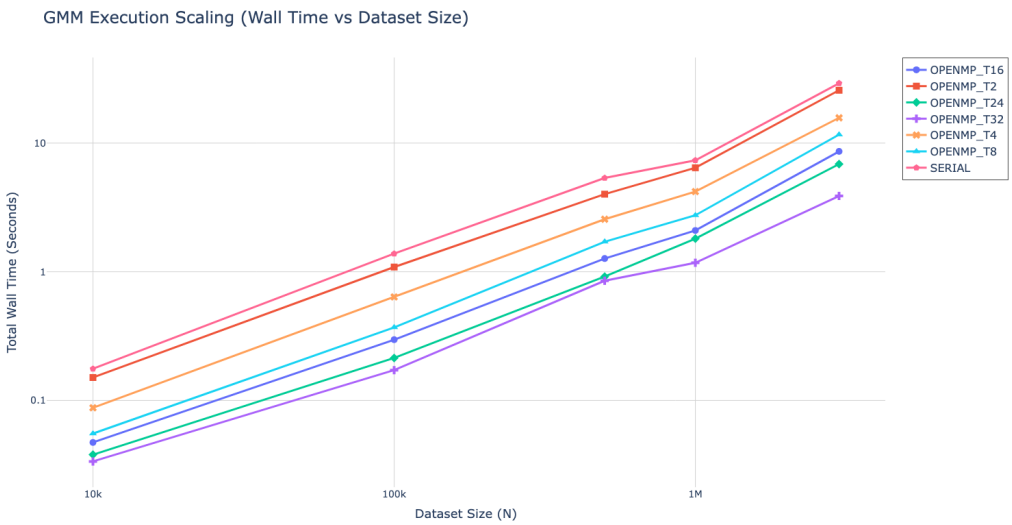

Figure 1: OpenMP performance scaling with varying thread counts ($D = 32, K = 8$).

8.2 Comparison of Serial, OpenMP, and CUDA ($D = 32, K = 8$)

Figure 2 presents a comparison between the serial implementation, the CUDA implementation, and the high-performance OpenMP configurations (T24 and T32) for the $D = 32, K = 8$ case.

Notably, for this specific workload, both the 24-thread and 32-thread OpenMP implementations significantly outperform the CUDA implementation. At $N = 3,000,000$, OpenMP T24 (6.87s) and T32 (3.88s) are both faster than CUDA (8.99s). This highlights that for moderate cluster counts, the CPU's AVX-512 vectorization and cache locality offer superior performance by avoiding the kernel launch overhead and atomic contention present in the GPU implementation.

8.3 CPU vs GPU Scaling Analysis (Broad Grid)

Our benchmarking results on a broader grid (varying K up to 96) reveal a significant performance "cross-over point" between the 32-thread OpenMP CPU implementation and the CUDA GPU implementation (Tesla P100). The experiments were conducted on a dataset of $N = 500,000$ points.

- **Low Complexity and Overhead:** For small values of K and D, the OpenMP implementation consistently outperforms CUDA. For instance, at $D = 4, K = 2$, OpenMP completes in 0.011s compared to CUDA's 0.098s. This is primarily due to the fixed overhead of CUDA kernel launches and the fact that the small model fits entirely within the CPU's L2/L3 caches.
- **The Cross-over Point:** As D and K increase, the workload transitions from being memory-latency bound to compute-throughput bound.
  - For $D = 8$, the cross-over occurs at $K = 8$, where CUDA begins to lead.
  - For $D = 32$, OpenMP holds its advantage until $K \approx 12$, after which CUDA's massive parallelism takes over (see Figure 3).

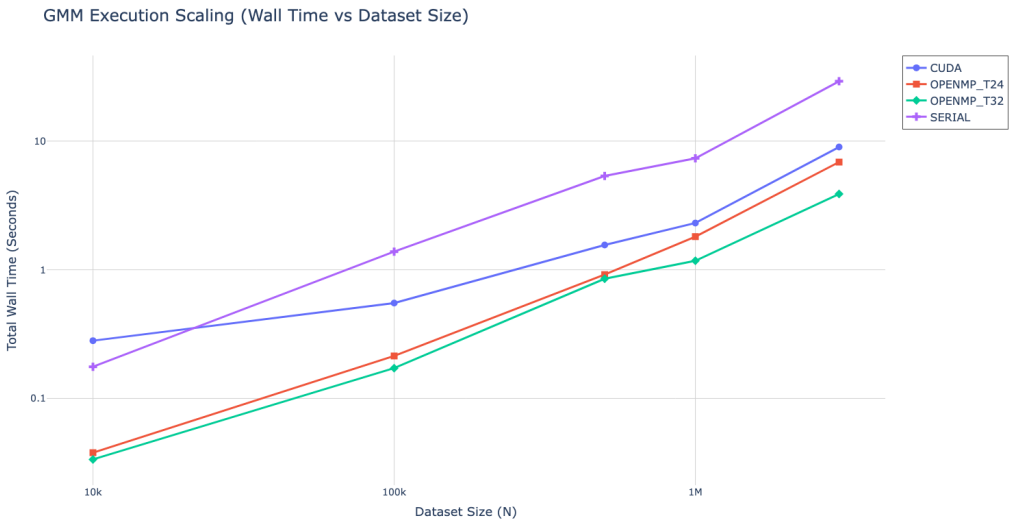

Figure 2: Comparative performance of Serial, CUDA, and high-thread OpenMP ($D = 32, K = 8$).

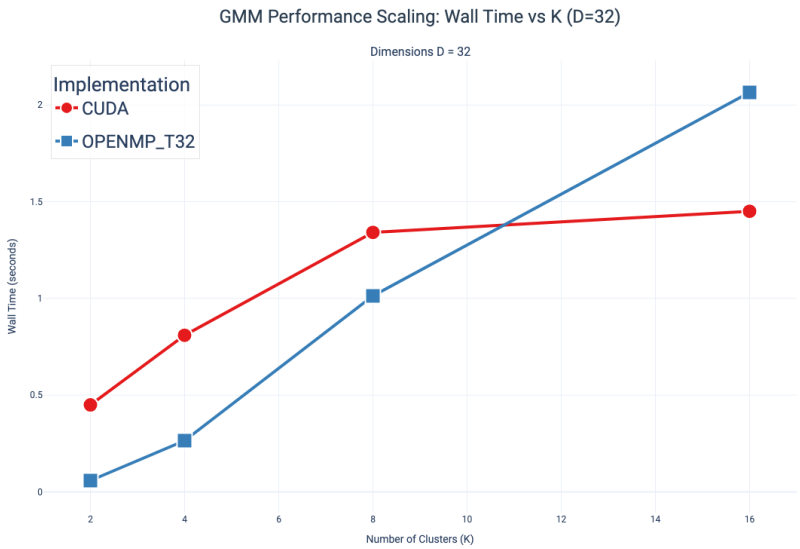

Figure 3: Performance cross-over between OpenMP T32 and CUDA ($D = 32$). OpenMP's efficiency at low K is eventually overtaken by CUDA's throughput as model complexity increases.

- **High Complexity Scaling:** At high cluster counts ($K \geq 32$), the GPU's superior throughput becomes dominant. At the maximum tested configuration ($D = 32, K = 96$), the CUDA implementation achieves a wall time of **9.3s**, outperforming the 32-thread OpenMP implementation (**25.5s**) by a factor of **2.7x**, and the serial reference (**88.8s**) by nearly **10x**.

8.4 Architectural Performance Factors

The observed scaling behavior is driven by several competing architectural factors:

- **Arithmetic Intensity:** At low K, the ratio of computation to memory access is low, making the GPU's memory bandwidth and synchronization overhead the limiting factors. As K increases, the $O(N \cdot K \cdot D^2)$ operations dominate, allowing the GPU to hide memory latency through massive thread concurrency.
- **CPU Vectorization Efficiency:** For $D = 32$, the CPU utilizes AVX-512 instructions to process the dimensionality loop with extreme efficiency. This keeps the CPU competitive even at moderate complexities.
- **Atomic Contention:** In the CUDA M-Step, 'atomicAdd' operations face high contention at low K as many threads attempt to update the same cluster parameters. As K increases, these updates are spread across more memory addresses, paradoxically reducing contention and improving the GPU's scaling efficiency.

8.5 3D Surface Analysis

To better visualize the global performance trends across the dimensionality (D) and cluster (K) space, we generated 3D surface plots of the wall time for Serial, OpenMP T32, and CUDA implementations.

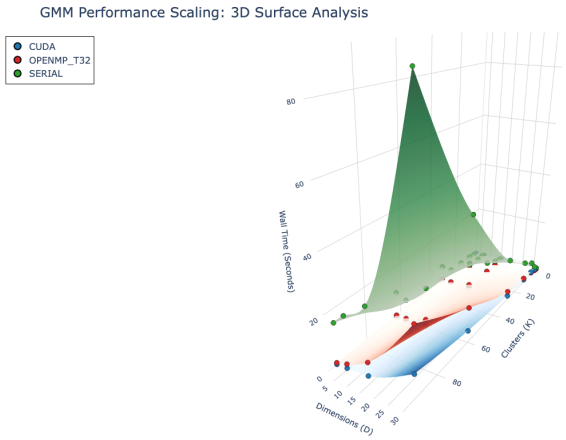

Figure 4: 3D Surface Analysis (Angle 1)

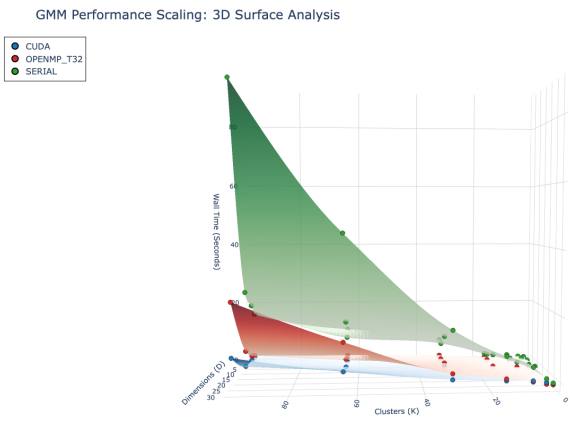

Figure 5: 3D Surface Analysis (Angle 2)

The surfaces clearly illustrate the diverging performance of the architectures. The Serial surface (green) rises sharply as both D and K increase. The OpenMP surface (red) remains relatively flat for a wider range of parameters, while the CUDA surface (blue) demonstrates its superior scaling at the highest complexities, maintaining the lowest wall times even as D and K reach their maximum values in our grid.

9 Future Work for GPU Acceleration

To further optimize the CUDA implementation and move the cross-over point to lower complexities, the following architectural redesigns are proposed:

1. **Kernel Fusion:** Combining the E-step and M-step kernels would reduce the number of passes over the $N \times D$ dataset from three to one, significantly easing the memory bandwidth bottleneck.

2. **GPU-Native Cholesky:** Offloading the Cholesky decomposition to cuSOLVER or a custom block-level kernel would eliminate the expensive iteration-blocking `cudaMemcpy` calls and CPU-GPU synchronization.

3. **Warp-Level Reductions:** Utilizing warp-shuffle instructions (`__shfl_down_sync`) for local reductions within the Streaming Multiprocessor would eliminate the reliance on `atomicAdd` in shared memory, further boosting throughput in the M-Step.

10 Conclusion

This project successfully demonstrated the fundamental principles determining High-Performance Computing (HPC) outcomes when porting complex machine learning models to parallel architectures. By implementing the Expectation-Maximization algorithm for Gaussian Mixture Models in both OpenMP and CUDA, critical intuitions surrounding hardware bottlenecks were quantified.

What worked exceptionally well was the OpenMP thread-local caching strategy combined with AVX-512 vector pipeline unrolling, which allowed the 32-thread CPU implementation to completely dominate memory-bound, low-complexity models (low D, low K). The L3 cache capabilities effectively masked main DRAM latencies in a way the GPU's PCIe bus transfers could not.

However, grid testing revealed absolute architectural limits. Once the dimensionality scalar forced expensive $\mathcal{O}(K \cdot D^3)$ Cholesky matrix inversions and massive cluster reductions simultaneously, the CPU pipelines were overwhelmed. Here, the raw floating-point strength of the Tesla P100 GPU breached the cross-over point. Despite heavy `atomicAdd` contention in shared memory and synchronization limits, the GPU delivered up to a $10\times$ speedup over the serial implementation under maximum tested stress limits ($D = 32, K = 96$).

Ultimately, this optimization exercise emphasized that parallel strategy cannot be one-size-fits-all. Crossover thresholds dictate the correct hardware target. Moving forward, eliminating memory movement penalties through advanced GPU Kernel Fusion and offloading Cholesky math to `cuSOLVER` remain the clearest paths to maximizing strict GPU supremacy across all model sizes.

References

[1] Dempster, A.P., Laird, N.M., Rubin, D.B. (1977). *Maximum Likelihood from Incomplete Data via the EM Algorithm. Journal of the Royal Statistical Society: Series B.*

[2] Bishop, C.M. (2006). *Pattern Recognition and Machine Learning.* Springer. (Chapter 9: Mixture Models and EM).

[3] NVIDIA Corporation. *NVIDIA CUDA C++ Programming Guide.* `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`

[4] scikit-learn developers. *GaussianMixture documentation.* `https://scikit-learn.org/stable/modules/mixture.html`

[5] Williams, S. et al. (2009). *Roofline: An Insightful Visual Performance Model. Communications of the ACM.*